



```
In [ ]: import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler, LabelEncoder
from google.colab import drive
import sys
import os
drive.mount('/content/drive')
drivePath = ('/content/drive/MyDrive/Visual Komputer Cerdas/Dataset/house
sys.path.append(drivePath)
```

Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", force_remount=True).

```
In [ ]: df = pd.read_table(drivePath, sep=';')
```

<ipython-input-65-7e9aba53a5f3>:1: DtypeWarning: Columns (2,3,4,5,6,7) have mixed types. Specify dtype option on import or set low_memory=False.
df = pd.read_table(drivePath, sep=';')

```
In [ ]: df.head(10)
```

Out[]:

	Date	Time	Global_active_power	Global_reactive_power	Voltage	Global_intensity	Sub
--	------	------	---------------------	-----------------------	---------	------------------	-----

0	16/12/2006	17:24:00	4.216	0.418	234.840	18.400	
1	16/12/2006	17:25:00	5.360	0.436	233.630	23.000	
2	16/12/2006	17:26:00	5.374	0.498	233.290	23.000	
3	16/12/2006	17:27:00	5.388	0.502	233.740	23.000	
4	16/12/2006	17:28:00	3.666	0.528	235.680	15.800	
5	16/12/2006	17:29:00	3.520	0.522	235.020	15.000	
6	16/12/2006	17:30:00	3.702	0.520	235.090	15.800	
7	16/12/2006	17:31:00	3.700	0.520	235.220	15.800	
8	16/12/2006	17:32:00	3.668	0.510	233.990	15.800	
9	16/12/2006	17:33:00	3.662	0.510	233.860	15.800	

1. Data Praprocessing

```
In [ ]: #Missing values
print("missing value :")
print(df.isna().sum())

#Cek duplikasi data
print("jumlah duplikasi = ", df.duplicated().sum())
```

```
missing value :
Date           0
Time           0
Global_active_power  0
Global_reactive_power  0
Voltage        0
Global_intensity  0
Sub_metering_1  0
Sub_metering_2  0
Sub_metering_3  25979
dtype: int64
jumlah duplikasi = 0
```

```
In [ ]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2075259 entries, 0 to 2075258
Data columns (total 9 columns):
#   Column                Dtype
---  -
0   Date                  object
1   Time                  object
2   Global_active_power    object
3   Global_reactive_power  object
4   Voltage               object
5   Global_intensity       object
6   Sub_metering_1         object
7   Sub_metering_2         object
8   Sub_metering_3         float64
dtypes: float64(1), object(8)
memory usage: 142.5+ MB
```

```
In [ ]: #Menghapus feature Date dan Time
df.drop(['Date', 'Time'], axis = 1, inplace = True)
```

```
In [ ]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2075259 entries, 0 to 2075258
Data columns (total 7 columns):
#   Column                Dtype
---  -
0   Global_active_power    object
1   Global_reactive_power  object
2   Voltage               object
3   Global_intensity       object
4   Sub_metering_1         object
5   Sub_metering_2         object
6   Sub_metering_3         float64
dtypes: float64(1), object(6)
memory usage: 110.8+ MB
```

```
In [ ]: #Menampilkan nilai unik
df['Global_active_power'].unique()
```

```
Out[ ]: array(['4.216', '5.360', '5.374', ..., 4.068, 3.94, 3.74], dtype=object)
```

```
In [ ]: #Membersihkan Data dari Tanda Petik dan Spasi Ekstra  
df['Global_active_power'] = df['Global_active_power'].str.replace("'", '')  
  
#Mengonversi Data ke Format Numerik dan Menangani Error  
df['Global_active_power'] = pd.to_numeric(df['Global_active_power'], errors='coerce')  
  
#Menampilkan Tipe Data dalam DataFrame  
print(df.dtypes)
```

```
Global_active_power      float64  
Global_reactive_power    object  
Voltage                  object  
Global_intensity         object  
Sub_metering_1           object  
Sub_metering_2           object  
Sub_metering_3           float64  
dtype: object
```

```
In [ ]: #Menampilkan nilai unik  
df['Global_reactive_power'].unique()
```

```

Out[ ]: array(['0.418', '0.436', '0.498', '0.502', '0.528', '0.522', '0.520',
               '0.510', '0.470', '0.478', '0.398', '0.422', '0.282', '0.152',
               '0.156', '0.000', '0.076', '0.090', '0.200', '0.058', '0.180',
               '0.144', '0.118', '0.108', '0.202', '0.192', '0.186', '0.116',
               '0.136', '0.148', '0.160', '0.158', '0.100', '0.082', '0.050',
               '0.052', '0.162', '0.086', '0.048', '0.054', '0.068', '0.166',
               '0.174', '0.178', '0.188', '0.088', '0.080', '0.070', '0.084',
               '0.074', '0.106', '0.092', '0.064', '0.046', '0.138', '0.134',
               '0.132', '0.120', '0.130', '0.154', '0.078', '0.142', '0.140',
               '0.146', '0.150', '0.170', '0.102', '0.072', '0.060', '0.056',
               '0.062', '0.112', '0.066', '0.172', '0.168', '0.194', '0.184',
               '0.096', '0.164', '0.182', '0.094', '0.098', '0.176', '0.190',
               '0.204', '0.220', '0.198', '0.208', '0.244', '0.334', '0.298',
               '0.296', '0.286', '0.278', '0.104', '0.258', '0.238', '0.256',
               '0.214', '0.230', '0.210', '0.240', '0.124', '0.122', '0.270',
               '0.302', '0.272', '0.250', '0.294', '0.300', '0.128', '0.126',
               '0.234', '0.242', '0.316', '0.280', '0.288', '0.224', '0.110',
               '0.248', '0.254', '0.222', '0.246', '0.216', '0.212', '0.114',
               '0.206', '0.228', '0.226', '0.196', '0.218', '0.368', '0.338',
               '0.310', '0.304', '0.292', '0.268', '0.266', '0.260', '0.396',
               '0.380', '0.400', '0.384', '0.416', '0.404', '0.382', '0.232',
               '0.314', '0.312', '0.262', '0.252', '0.236', '0.348', '0.342',
               '0.340', '0.350', '0.504', '0.344', '0.346', '0.366', '0.574',
               '0.582', '0.370', '0.364', '0.360', '0.362', '0.372', '0.392',
               '0.354', '0.356', '0.358', '0.402', '0.394', '0.390', '0.386',
               '0.274', '0.264', '0.324', '0.352', '0.276', '0.306', '0.376',
               '0.562', '0.440', '0.378', '0.426', '0.450', '0.374', '0.388',
               '0.284', '0.290', '0.308', '0.318', '0.320', '0.322', '0.326',
               '0.336', '0.414', '0.408', '0.472', '0.464', '0.462', '0.578',
               '0.576', '0.580', '0.474', '0.514', '0.646', '0.636', '0.598',
               '0.586', '0.566', '0.406', '0.430', '0.412', '0.442', '0.452',
               '0.432', '0.330', '0.328', '0.538', '0.564', '0.552', '0.556',
               '0.606', '0.424', '0.492', '0.526', '0.512', '0.482', '0.446',
               '0.332', '?', '0.548', '0.534', '0.532', '0.688', '0.664', '0.66
0',
               '0.530', '0.516', '0.572', '0.540', '0.614', '0.568', '0.444',
               '0.456', '0.448', '0.480', '0.434', '0.438', '0.494', '0.496',
               '0.468', '0.560', '0.622', '0.570', '0.618', '0.624', '0.466',
               '0.420', '0.500', '0.602', '0.632', '0.724', '0.706', '0.484',
               '0.458', '0.428', '0.486', '0.698', '0.708', '0.686', '0.666',
               '0.658', '0.490', '0.454', '0.640', '0.590', '0.524', '0.634',
               '0.476', '0.694', '0.700', '0.794', '0.592', '0.460', '0.656',
               '0.644', '0.488', '0.410', '0.550', '0.616', '0.800', '0.790',
               '0.788', '0.670', '0.506', '0.600', '0.680', '0.678', '0.682',
               '0.690', '0.508', '0.604', '0.608', '0.554', '0.518', '0.650',
               '0.648', '0.642', '0.654', '0.754', '0.750', '0.742', '0.594',
               '0.536', '0.544', '0.630', '0.542', '0.672', '0.588', '0.596',
               '0.584', '0.674', '0.726', '0.768', '0.774', '0.712', '0.856',
               '0.862', '0.848', '0.730', '0.710', '0.662', '0.722', '0.704',
               '0.546', '0.638', '0.628', '0.620', '0.716', '0.778', '0.702',
               '0.676', '0.834', '0.802', '0.772', '0.558', '0.626', '0.652',
               '0.760', '0.780', '0.784', '0.786', '0.792', '0.782', '0.832',
               '0.842', '0.744', '0.822', '0.612', '0.692', '0.684', '0.874',
               '0.804', '0.696', '0.610', '0.718', '0.732', '0.720', '0.748',
               '0.796', '0.736', '0.820', '0.808', '0.756', '0.776', '0.806',

```

	'0.764', '0.818', '0.770', '0.758', '0.858', '0.860', '0.850',
	'0.752', '0.714', '0.840', '0.830', '0.838', '0.738', '0.828',
	'0.740', '0.746', '0.988', '0.966', '0.960', '0.956', '0.854',
	'0.912', '0.734', '0.728', '0.826', '0.880', '0.878', '0.876',
	'0.930', '0.986', '0.972', '0.946', '0.902', '1.046', '1.148',
	'1.054', '0.890', '0.668', '1.092', '0.996', '0.974', '1.040',
	'1.060', '1.028', '0.908', '0.898', '0.950', '0.964', '0.940',
	'0.928', '0.896', '0.868', '0.870', '0.866', '0.864', '0.872',
	'0.812', '0.836', '0.824', '0.814', '0.762', '0.994', '1.032',
	'0.816', '0.944', '0.968', '0.916', '1.014', '1.008', '1.002',
	'0.810', '1.000', '1.006', '1.012', '1.034', '0.852', '0.924',
	'0.942', '0.982', '0.906', '0.932', '0.914', '0.846', '0.844',
	'1.222', '1.190', '1.180', '1.118', '1.082', '1.080', '0.910',
	'1.102', '1.274', '1.364', '1.390', '0.918', '1.198', '1.192',
	'0.936', '0.954', '0.980', '0.990', '0.904', '0.798', '0.766',
	'1.090', '1.016', '0.922', '1.036', '0.892', '0.894', '0.900',
	'0.886', '1.096', '0.920', '0.962', '0.958', '0.998', '1.044',
	'1.048', '0.938', '0.934', '1.130', '1.218', '0.888', '0.882',
	'1.156', '1.024', '1.088', '1.084', '1.076', '1.066', '1.070',
	'1.240', '0.926', '0.952', '0.884', '1.186', '1.108', '1.030',
	'1.020', '0.978', '0.970', '0.948', '1.064', '1.078', '1.074',
	'1.124', '1.018', '0.976', '1.038', '1.094', '1.100', '1.026',
0.0,	0.084, 0.086, 0.088, 0.082, 0.07, 0.062, 0.064, 0.06, 0.066, 0.04
6,	0.076, 0.078, 0.048, 0.08, 0.068, 0.072, 0.16, 0.188, 0.102, 0.21
6,	0.214, 0.212, 0.094, 0.058, 0.334, 0.408, 0.4, 0.47, 0.56, 0.634,
	0.638, 0.64, 0.65, 0.656, 0.574, 0.242, 0.232, 0.23, 0.15, 0.074,
	0.22, 0.278, 0.322, 0.32, 0.324, 0.254, 0.204, 0.154, 0.218, 0.11
2,	0.2, 0.196, 0.126, 0.056, 0.05, 0.052, 0.092, 0.09, 0.054, 0.114,
	0.116, 0.11, 0.108, 0.104, 0.1, 0.106, 0.096, 0.098, 0.146, 0.33
2,	0.314, 0.236, 0.54, 0.466, 0.458, 0.424, 0.33, 0.328, 0.318, 0.29
8,	0.302, 0.306, 0.3, 0.29, 0.198, 0.206, 0.122, 0.21, 0.24, 0.312,
	0.31, 0.316, 0.296, 0.19, 0.194, 0.144, 0.304, 0.398, 0.454, 0.44
4,	0.376, 0.374, 0.378, 0.294, 0.138, 0.336, 0.342, 0.34, 0.338,
	0.244, 0.228, 0.234, 0.222, 0.308, 0.286, 0.202, 0.208, 0.17,
	0.136, 0.266, 0.264, 0.268, 0.256, 0.26, 0.262, 0.258, 0.25, 0.1
8,	0.134, 0.284, 0.344, 0.348, 0.174, 0.248, 0.246, 0.238, 0.226,
	0.164, 0.224, 0.184, 0.142, 0.132, 0.128, 0.148, 0.13, 0.124, 0.1
2,	0.158, 0.252, 0.272, 0.192, 0.182, 0.288, 0.382, 0.166, 0.162,
	0.118, 0.186, 0.406, 0.478, 0.156, 0.14, 0.152, 0.326, 0.366,
0.5,	0.506, 0.514, 0.518, 0.524, 0.52, 0.528, 0.526, 0.522, 0.178,
	0.176, 0.168, 0.172, 0.422, 0.556, 0.492, 0.426, 0.558, 0.776,
	0.87, 0.83, 0.744, 0.486, 0.396, 0.362, 0.356, 0.358, 0.372, 0.3
7,	0.276, 0.274, 0.282, 0.27, 0.412, 0.472, 0.438, 0.434, 0.436,
	0.428, 0.364, 0.414, 0.55, 0.652, 0.646, 0.644, 0.648, 0.554,

```

0.392, 0.368, 0.354, 0.394, 0.39, 0.292, 0.826, 0.912, 0.9, 1.01
2,
0.442, 0.828, 0.956, 0.914, 1.066, 1.108, 0.894, 0.844, 0.758,
0.756, 0.468, 0.72, 0.738, 0.734, 0.552, 0.534, 0.388, 0.474,
0.622, 0.642, 0.624, 0.626, 0.35, 0.494, 0.46, 0.38, 0.36, 0.548,
0.542, 0.546, 0.544, 0.504, 0.41, 0.402, 0.28, 0.416, 0.432, 0.4
3,
0.564, 0.404, 0.456, 0.464, 0.532, 0.6, 0.598, 0.576, 0.578, 0.58
2,
0.462, 0.476, 0.508, 0.352, 0.512, 0.51, 0.516, 0.418, 0.42, 0.53
6,
0.384, 0.452, 0.448, 0.45, 0.386, 0.346, 0.572, 0.654, 0.662,
0.658, 0.664, 0.668, 0.636, 0.606, 0.77, 0.618, 0.82, 0.818, 0.7
1,
0.588, 0.58, 0.584, 0.496, 0.502, 0.482, 0.48, 0.498, 0.488, 0.63
2,
0.484, 0.568, 0.566, 0.614, 0.66, 0.762, 0.764, 0.628, 0.604,
0.686, 0.7, 0.698, 0.704, 0.49, 0.84, 0.834, 0.726, 0.716, 0.708,
0.706, 0.712, 0.44, 0.538, 0.732, 0.674, 0.562, 0.666, 0.586, 0.6
7,
0.672, 0.728, 0.74, 0.62, 0.79, 0.592, 0.57, 0.602, 0.722, 0.724,
0.68, 0.59, 0.594, 0.808, 0.802, 0.772, 0.446, 0.684, 0.678, 0.79
6,
0.798, 0.696, 0.692, 0.69], dtype=object)

```

```

In [ ]: #Membersihkan Data dari Tanda Petik dan Spasi Ekstra
df['Global_reactive_power'] = df['Global_reactive_power'].str.replace('\"')

#Mengonversi Data ke Format Numerik dan Menangani Error
df['Global_reactive_power'] = pd.to_numeric(df['Global_reactive_power'],

#Menampilkan Tipe Data dalam DataFrame
print(df.dtypes)

```

```

Global_active_power    float64
Global_reactive_power   float64
Voltage                 object
Global_intensity        object
Sub_metering_1          object
Sub_metering_2          object
Sub_metering_3          float64
dtype: object

```

```

In [ ]: #Menampilkan nilai unik
df['Voltage'].unique()

```

```

Out[ ]: array(['234.840', '233.630', '233.290', ..., 228.57, 229.09, 230.31],
          dtype=object)

```

```

In [ ]: #Membersihkan Data dari Tanda Petik dan Spasi Ekstra
df['Voltage'] = df['Voltage'].str.replace('\"', '').str.strip()

#Mengonversi Data ke Format Numerik dan Menangani Error
df['Voltage'] = pd.to_numeric(df['Voltage'], errors='coerce')

```

```
#Menampilkan Tipe Data dalam DataFrame  
print(df.dtypes)
```

```
Global_active_power    float64  
Global_reactive_power  float64  
Voltage                float64  
Global_intensity       object  
Sub_metering_1         object  
Sub_metering_2         object  
Sub_metering_3         float64  
dtype: object
```

```
In [ ]: #Menampilkan nilai unik  
df['Global_intensity'].unique()
```



```

Out[ ]: array(['18.400', '23.000', '15.800', '15.000', '19.600', '23.200',
               '22.400', '22.600', '17.600', '14.200', '13.800', '14.400',
               '16.400', '25.400', '33.200', '30.600', '22.000', '19.400',
               '13.600', '13.400', '11.600', '19.200', '10.400', '11.800',
               '11.000', '21.000', '26.200', '29.000', '27.800', '27.000',
               '19.000', '9.600', '10.000', '21.400', '17.800', '12.000',
               '12.400', '15.200', '20.800', '15.600', '26.400', '18.800',
               '13.000', '9.800', '9.200', '13.200', '18.000', '10.800', '10.60
0',
               '18.200', '17.000', '17.400', '14.800', '14.600', '15.400',
               '14.000', '8.400', '9.400', '7.800', '8.200', '8.600', '11.400',
               '12.600', '9.000', '7.600', '12.800', '7.400', '5.600', '4.600',
               '7.200', '1.600', '2.400', '2.800', '1.200', '4.400', '4.000',
               '10.200', '6.800', '5.200', '6.000', '6.600', '8.000', '2.000',
               '4.800', '5.400', '16.600', '3.000', '1.800', '8.800', '1.400',
               '21.800', '20.200', '16.000', '18.600', '16.800', '11.200',
               '2.600', '16.200', '4.200', '7.000', '3.800', '3.200', '5.000',
               '3.600', '0.800', '5.800', '1.000', '6.400', '6.200', '3.400',
               '2.200', '12.200', '30.000', '29.400', '28.200', '25.800',
               '20.000', '24.800', '24.600', '25.200', '25.000', '23.400',
               '23.800', '20.600', '21.600', '19.800', '20.400', '26.600',
               '17.200', '21.200', '23.600', '29.200', '33.600', '28.000',
               '30.200', '33.400', '24.200', '24.000', '22.200', '24.400',
               '22.800', '?', '30.400', '34.200', '32.800', '33.000', '32.600',
               '31.600', '28.600', '25.600', '31.000', '28.400', '27.200',
               '29.800', '29.600', '37.000', '36.800', '32.400', '27.400',
               '35.000', '34.800', '32.200', '26.000', '35.200', '36.200',
               '27.600', '26.800', '34.400', '31.400', '30.800', '31.200',
               '39.000', '39.200', '39.400', '28.800', '38.800', '35.600',
               '32.000', '33.800', '36.000', '37.200', '34.000', '31.800',
               '36.600', '40.400', '37.400', '38.200', '36.400', '35.400',
               '34.600', '35.800', '38.000', '40.200', '41.000', '41.400',
               '41.200', '41.800', '38.600', '44.400', '46.400', '43.200',
               '39.800', '40.800', '0.600', '0.400', '37.800', '40.600', '38.40
0',
               '42.000', '42.200', '39.600', '42.600', '44.200', '37.600',
               '40.000', '0.200', '44.600', '43.400', '43.000', '45.800',
               '48.400', 2.6, 2.4, 2.2, 1.6, 1.4, 1.8, 2.0, 4.0, 4.4, 5.0, 5.4,
               5.6, 5.8, 2.8, 3.0, 3.2, 6.0, 6.2, 6.4, 6.6, 3.4, 7.8, 7.4, 8.0,
               4.2, 1.2, 1.0, 13.6, 15.6, 9.0, 3.8, 4.8, 4.6, 3.6, 7.0, 6.8, 1
1.8,
               10.8, 10.2, 10.4, 10.6, 10.0, 9.8, 8.4, 8.2, 8.6, 9.4, 11.0, 1
1.2,
               11.6, 7.2, 8.8, 9.2, 7.6, 16.6, 16.2, 15.8, 9.6, 15.4, 14.2, 1
8.6,
               18.8, 16.0, 16.4, 11.4, 12.0, 14.8, 13.2, 14.0, 19.0, 22.4, 22.2,
               20.6, 14.6, 13.4, 12.2, 12.4, 12.6, 13.8, 12.8, 14.4, 15.2, 5.2,
               17.0, 17.8, 21.6, 15.0, 20.8, 17.2, 16.8, 13.0, 19.2, 21.4, 19.8,
               19.4, 20.0, 18.0, 17.6, 18.4, 20.4, 20.2, 21.0, 19.6, 22.0, 23.4,
               24.6, 24.8, 25.8, 25.0, 25.2, 25.4, 26.4, 34.0, 33.8, 33.0, 23.6,
               25.6, 24.4, 17.4, 23.8, 24.2, 18.2, 21.2, 23.0, 27.8, 28.8, 22.8,
               26.8, 21.8, 22.6, 26.6, 27.0, 28.0, 28.2, 24.0, 29.4, 31.6, 33.2,
               30.4, 30.6, 31.0, 37.2, 43.0, 42.8, 38.2, 27.2, 29.2, 30.2, 29.6,
               27.6, 30.0, 29.0, 28.4, 27.4, 23.2, 26.0], dtype=object)

```

```
In [ ]: #Membersihkan Data dari Tanda Petik dan Spasi Ekstra
df['Global_intensity'] = df['Global_intensity'].str.replace("'", '').str.strip()

#Mengonversi Data ke Format Numerik dan Menangani Error
df['Global_intensity'] = pd.to_numeric(df['Global_intensity'], errors='coerce')

#Menampilkan Tipe Data dalam DataFrame
print(df.dtypes)
```

```
Global_active_power    float64
Global_reactive_power  float64
Voltage                float64
Global_intensity        float64
Sub_metering_1          object
Sub_metering_2          object
Sub_metering_3          float64
dtype: object
```

```
In [ ]: #Menampilkan nilai unik
df['Sub_metering_1'].unique()
```

```
Out[ ]: array(['0.000', '1.000', '2.000', '6.000', '37.000', '36.000', '38.000',
              '25.000', '12.000', '11.000', '9.000', '4.000', '33.000', '5.000',
              '0',
              '34.000', '8.000', '26.000', '27.000', '15.000', '20.000',
              '21.000', '23.000', '16.000', '10.000', '39.000', '32.000', '?',
              '19.000', '35.000', '30.000', '40.000', '7.000', '31.000',
              '17.000', '3.000', '29.000', '24.000', '14.000', '41.000',
              '53.000', '76.000', '75.000', '77.000', '74.000', '28.000',
              '68.000', '58.000', '62.000', '13.000', '18.000', '22.000',
              '64.000', '71.000', '56.000', '72.000', '45.000', '42.000',
              '59.000', '66.000', '43.000', '44.000', '48.000', '46.000',
              '73.000', '55.000', '60.000', '70.000', '63.000', '67.000',
              '47.000', '65.000', '51.000', '50.000', '69.000', '78.000',
              '57.000', '49.000', '61.000', '52.000', '54.000', '79.000',
              '80.000', '81.000', '82.000', '83.000', '84.000', '88.000',
              '86.000', '87.000', 0.0, 1.0, 2.0, 40.0, 39.0, 9.0, 10.0, 34.0,
              25.0, 38.0, 37.0, 23.0, 33.0, 32.0, 27.0, 24.0, 22.0, 21.0, 20.0,
              18.0, 19.0, 13.0, 31.0, 29.0, 17.0, 16.0, 15.0, 14.0, 3.0, 42.0,
              12.0, 28.0, 36.0, 47.0, 48.0, 5.0, 35.0, 4.0, 6.0, 11.0, 53.0,
              54.0, 52.0, 43.0, 63.0, 76.0, 80.0, 81.0, 72.0, 60.0, 79.0, 70.0,
              45.0, 30.0, 44.0, 46.0, 8.0, 26.0, 49.0, 7.0, 51.0, 41.0, 50.0,
              57.0], dtype=object)
```

```
In [ ]: #Membersihkan Data dari Tanda Petik dan Spasi Ekstra
df['Sub_metering_1'] = df['Sub_metering_1'].str.replace("'", '').str.strip()

#Mengonversi Data ke Format Numerik dan Menangani Error
df['Sub_metering_1'] = pd.to_numeric(df['Sub_metering_1'], errors='coerce')

#Menampilkan Tipe Data dalam DataFrame
print(df.dtypes)
```

```
Global_active_power    float64
Global_reactive_power  float64
Voltage                float64
Global_intensity       float64
Sub_metering_1         float64
Sub_metering_2         object
Sub_metering_3         float64
dtype: object
```

```
In [ ]: #Menampilkan nilai unik
df['Sub_metering_2'].unique()
```

```
Out[ ]: array(['1.000', '2.000', '0.000', '16.000', '37.000', '36.000', '22.000',
               '12.000', '27.000', '20.000', '9.000', '4.000', '3.000', '5.000',
               '13.000', '35.000', '28.000', '26.000', '24.000', '21.000',
               '17.000', '33.000', '25.000', '23.000', '18.000', '30.000',
               '31.000', '7.000', '38.000', '29.000', '50.000', '71.000',
               '59.000', '62.000', '70.000', '40.000', '34.000', '8.000',
               '15.000', '6.000', '32.000', '19.000', '14.000', '49.000',
               '72.000', '73.000', '61.000', '39.000', '42.000', '10.000',
               '56.000', '11.000', '?', '43.000', '67.000', '63.000', '53.000',
               '65.000', '52.000', '69.000', '48.000', '74.000', '41.000',
               '47.000', '58.000', '68.000', '64.000', '75.000', '46.000',
               '57.000', '60.000', '66.000', '44.000', '78.000', '77.000',
               '76.000', '51.000', '55.000', '45.000', '54.000', '79.000',
               '80.000', 0.0, 1.0, 2.0, 36.0, 40.0, 39.0, 41.0, 23.0, 10.0, 6.0,
               5.0, 18.0, 37.0, 28.0, 38.0, 9.0, 13.0, 26.0, 31.0, 30.0, 24.0,
               4.0, 3.0, 16.0, 32.0, 12.0, 35.0, 56.0, 64.0, 73.0, 72.0, 74.0,
               75.0, 71.0, 21.0, 8.0, 20.0, 7.0, 33.0, 34.0, 60.0, 22.0, 19.0,
               42.0, 43.0, 69.0, 49.0, 27.0, 65.0, 62.0, 25.0, 17.0, 11.0, 70.0,
               63.0, 29.0, 54.0, 61.0, 68.0, 66.0, 67.0, 51.0, 15.0], dtype=object)
```

```
In [ ]: #Membersihkan Data dari Tanda Petik dan Spasi Ekstra
df['Sub_metering_2'] = df['Sub_metering_2'].str.replace('"', '').str.strip()

#Mengonversi Data ke Format Numerik dan Menangani Error
df['Sub_metering_2'] = pd.to_numeric(df['Sub_metering_2'], errors='coerce')

#Menampilkan Tipe Data dalam DataFrame
print(df.dtypes)
```

```
Global_active_power    float64
Global_reactive_power  float64
Voltage                float64
Global_intensity       float64
Sub_metering_1         float64
Sub_metering_2         float64
Sub_metering_3         float64
dtype: object
```

```
In [ ]: #Missing values
print("missing value :")
print(df.isna().sum())
```

```
#Cek duplikasi data
print("jumlah duplikasi = ", df.duplicated().sum())
```

```
missing value :
Global_active_power      69622
Global_reactive_power    69622
Voltage                  69622
Global_intensity         69622
Sub_metering_1           69622
Sub_metering_2           69622
Sub_metering_3           25979
dtype: int64
jumlah duplikasi = 209948
```

In []: df

Out[]:

	Global_active_power	Global_reactive_power	Voltage	Global_intensity	Sub_metering
0	4.216	0.418	234.84	18.4	
1	5.360	0.436	233.63	23.0	
2	5.374	0.498	233.29	23.0	
3	5.388	0.502	233.74	23.0	
4	3.666	0.528	235.68	15.8	
...	...	...	...	...	
2075254	NaN	NaN	NaN	NaN	
2075255	NaN	NaN	NaN	NaN	
2075256	NaN	NaN	NaN	NaN	
2075257	NaN	NaN	NaN	NaN	
2075258	NaN	NaN	NaN	NaN	

2075259 rows × 7 columns

```
In [ ]: #Menghapus baris yang ada nilai hilangnya (NaN)
df.dropna(inplace=True)
```

```
In [ ]: #Menghapus duplikasi
df.drop_duplicates(inplace=True)
```

```
In [ ]: #Missing values
print("missing value :")
print(df.isna().sum())

#Cek duplikasi data
print("jumlah duplikasi = ", df.duplicated().sum())
```

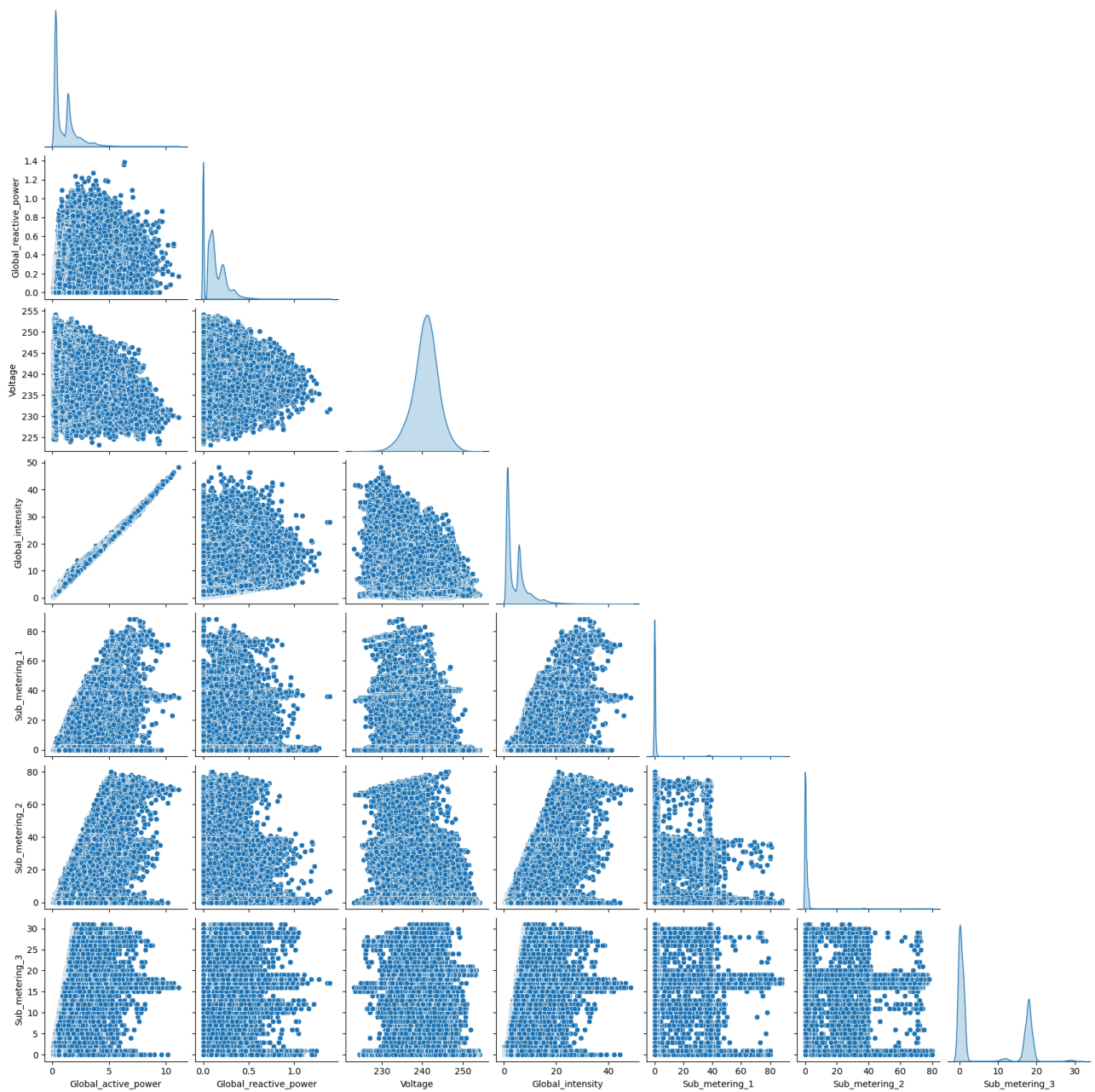
```
missing value :
Global_active_power      0
Global_reactive_power    0
Voltage                  0
Global_intensity         0
Sub_metering_1           0
Sub_metering_2           0
Sub_metering_3           0
dtype: int64
jumlah duplikasi = 0
```

```
In [ ]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 1865278 entries, 0 to 2031615
Data columns (total 7 columns):
#   Column                Dtype
---  -
0   Global_active_power    float64
1   Global_reactive_power  float64
2   Voltage                float64
3   Global_intensity       float64
4   Sub_metering_1         float64
5   Sub_metering_2         float64
6   Sub_metering_3         float64
dtypes: float64(7)
memory usage: 113.8 MB
```

- Sebaran Data

```
In [ ]: #Membuat pairplot
sns.pairplot(df, corner=True, diag_kind="kde")
plt.show()
```



```
In [ ]: df.head()
```

```
Out[ ]:
```

	Global_active_power	Global_reactive_power	Voltage	Global_intensity	Sub_metering_1	Sub_metering_2	Sub_metering_3
0	4.216	0.418	234.84	18.4	0.0	0.0	0.0
1	5.360	0.436	233.63	23.0	0.0	0.0	0.0
2	5.374	0.498	233.29	23.0	0.0	0.0	0.0
3	5.388	0.502	233.74	23.0	0.0	0.0	0.0
4	3.666	0.528	235.68	15.8	0.0	0.0	0.0

In []: *#Membuat grafik*

```
import seaborn as sns
import matplotlib.pyplot as plt

#Nilai maksimum sub_metering
max_values = {
    "Sub_metering_1": df["Sub_metering_1"].max(), #nilai maks dari Sub_m
    "Sub_metering_2": df["Sub_metering_2"].max(), #nilai maks dari Sub_m
    "Sub_metering_3": df["Sub_metering_3"].max()  #nilai maks dari Sub_m
}

#Membuat bar plot
plt.figure(figsize=(7,5))
ax = sns.barplot(x=list(max_values.keys()), y=list(max_values.values()),

#Menambahkan label di atas setiap batang
for p in ax.patches:
    ax.annotate(f'{p.get_height():.1f}', #menampilkan nilai maksimum
                (p.get_x() + p.get_width() / 2, p.get_height()),
                ha='center', va='bottom', fontsize=12)

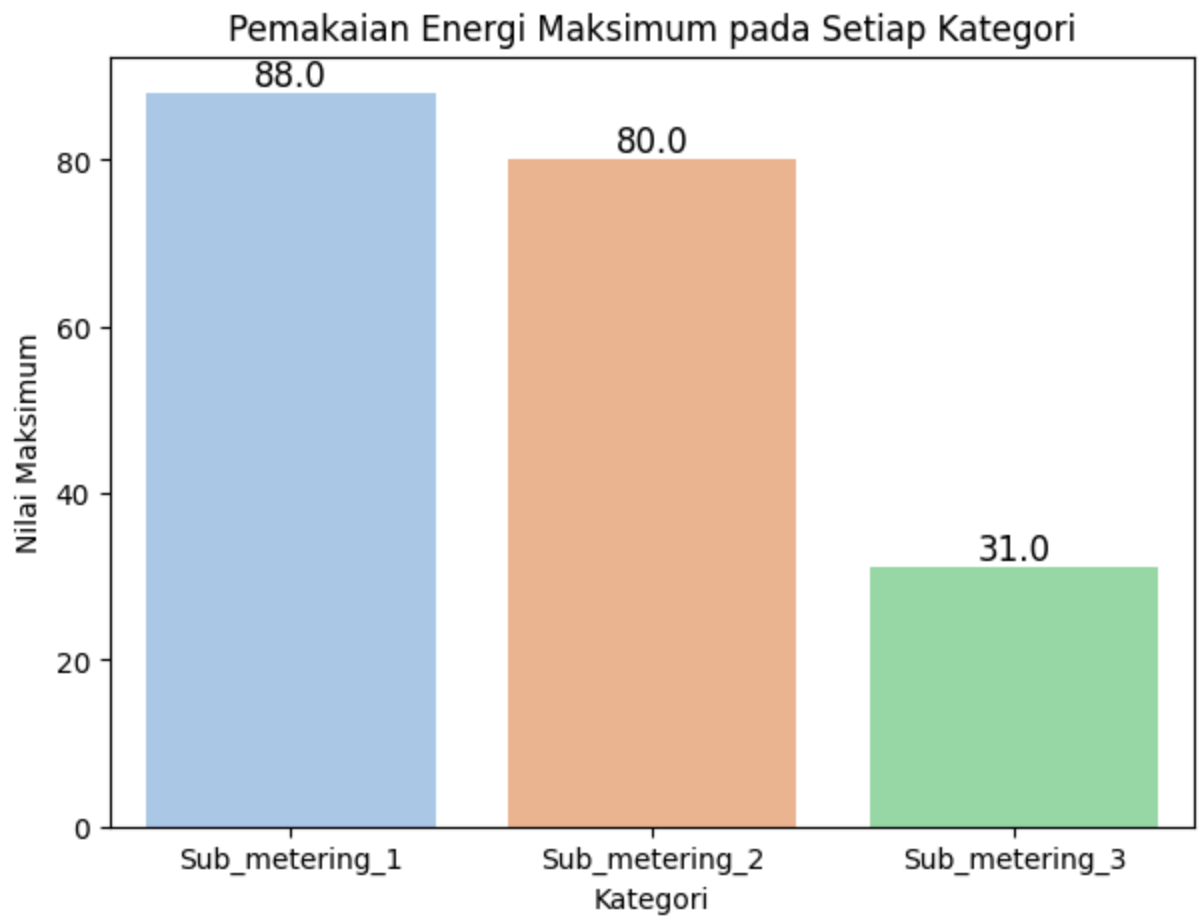
#Menambahkan label untuk sumbu X dan Y serta judul grafiknya
plt.xlabel("Kategori")
plt.ylabel("Nilai Maksimum")
plt.title("Pemakaian Energi Maksimum pada Setiap Kategori")

plt.show()
```

<ipython-input-93-0b21a30543f7>:15: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
ax = sns.barplot(x=list(max_values.keys()), y=list(max_values.values()), palette="pastel")
```

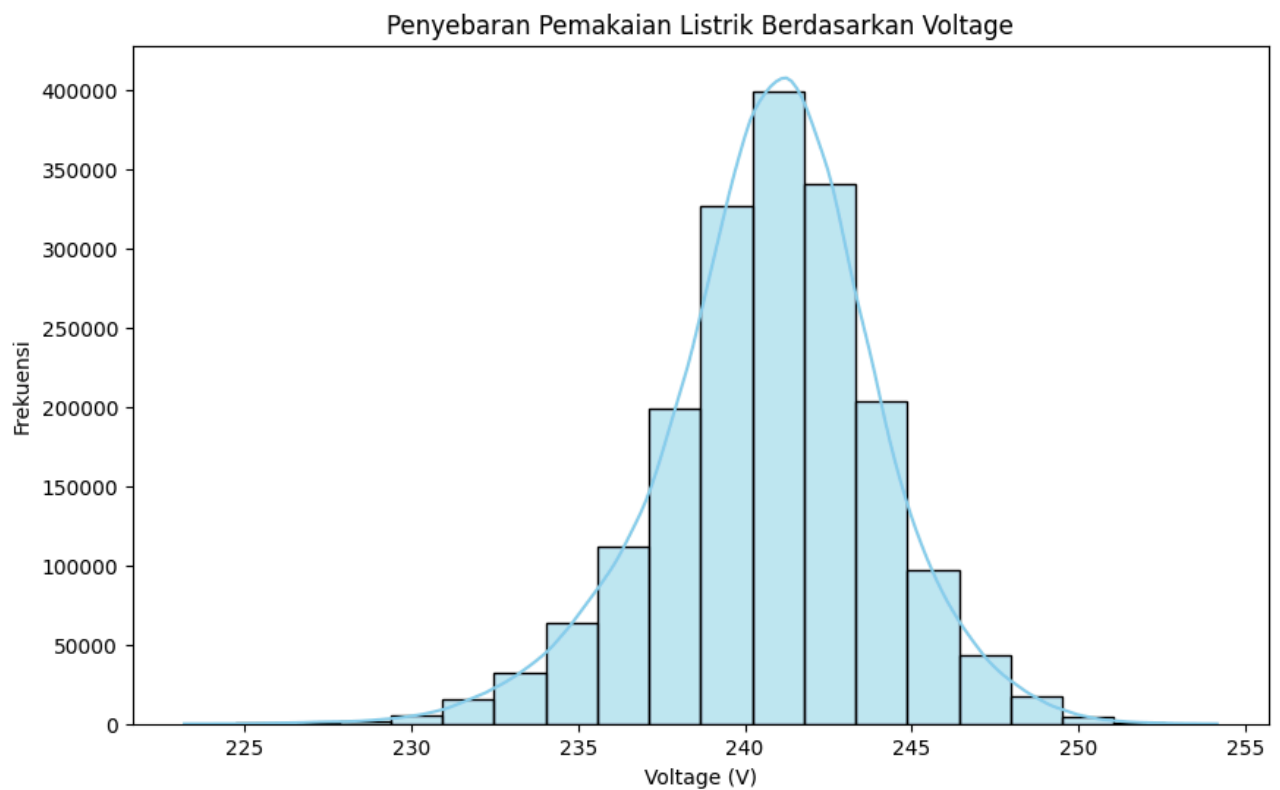


In []: *#Membuat grafik*

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(10,6))
ax = sns.histplot(df["Voltage"], bins=20, kde=True, color="skyblue")
ax.set_xlabel("Voltage (V)")
ax.set_ylabel("Frekuensi")
ax.set_title("Penyebaran Pemakaian Listrik Berdasarkan Voltage")

plt.show()
```

```
In [ ]: df.describe()
```

```
Out[ ]:
```

	Global_active_power	Global_reactive_power	Voltage	Global_intensity	Sub_meter
count	1.865278e+06	1.865278e+06	1.865278e+06	1.865278e+06	1.865278e+06
mean	1.147987e+00	1.319954e-01	2.407450e+02	4.869764e+00	1.204250e+00
std	1.071512e+00	1.122850e-01	3.242181e+00	4.500438e+00	6.370200e+00
min	7.600000e-02	0.000000e+00	2.232000e+02	2.000000e-01	0.000000e+00
25%	3.260000e-01	5.800000e-02	2.388800e+02	1.400000e+00	0.000000e+00
50%	7.380000e-01	1.060000e-01	2.409100e+02	3.200000e+00	0.000000e+00
75%	1.574000e+00	2.020000e-01	2.428000e+02	6.600000e+00	0.000000e+00
max	1.112200e+01	1.390000e+00	2.541500e+02	4.840000e+01	8.800000e+00

```
In [ ]: #Scaling data (normalisasi data)
scaler = StandardScaler()

#Proses scaling
df = pd.DataFrame(scaler.fit_transform(df), columns = df.columns)
df.head(5)
```

Out[]:

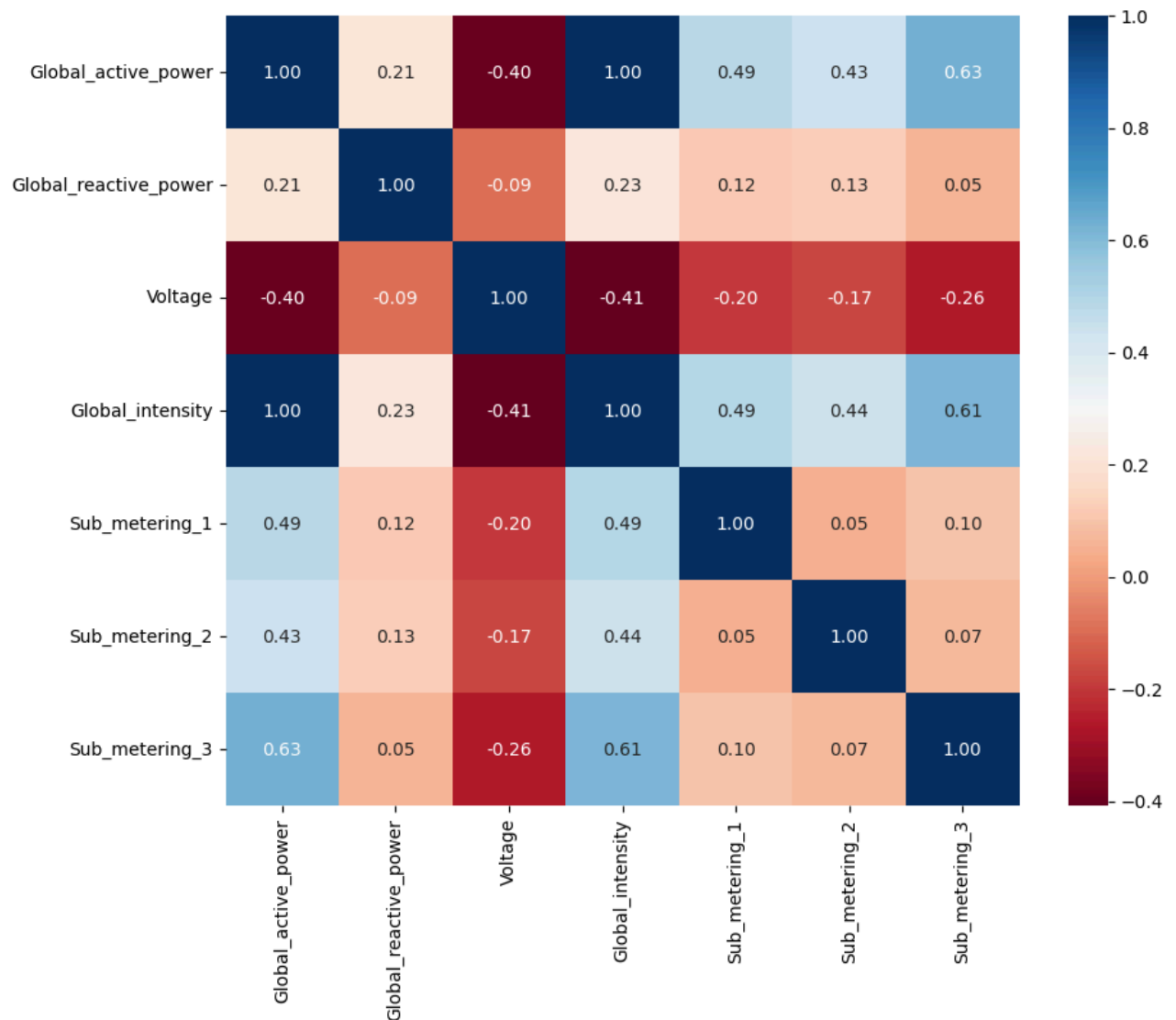
	Global_active_power	Global_reactive_power	Voltage	Global_intensity	Sub_metering_1	S
--	---------------------	-----------------------	---------	------------------	----------------	---

0	2.863258	2.547131	-1.821292	3.006427	-0.189045	
1	3.930908	2.707437	-2.194498	4.028550	-0.189045	
2	3.943974	3.259603	-2.299366	4.028550	-0.189045	
3	3.957040	3.295227	-2.160570	4.028550	-0.189045	
4	2.349964	3.526781	-1.562207	2.428706	-0.189045	

In []: *#Menganalisis korelasi antar fitur dalam dataset*

```
import seaborn as sns
import matplotlib.pyplot as plt

corr_matrix = df.corr()
plt.figure(figsize=(10, 8))
sns.heatmap(corr_matrix, annot=True, cmap='RdBu', fmt='.2f')
plt.show()
```



Berdasarkan heatmap korelasi, **Global_active_power** memiliki hubungan kuat dengan **Global_intensity (1.00)** dan **Sub_metering_3 (0.63)**, menunjukkan bahwa daya aktif dan intensitas arus berperan penting dalam konsumsi listrik. Sebaliknya, **Voltage berkorelasi negatif dengan Global_active_power (-0.40) dan Global_intensity (-0.41)**, menandakan tegangan cenderung menurun saat daya atau arus meningkat.

Global_reactive_power serta Sub_metering_1 dan Sub_metering_2 memiliki korelasi lemah, sehingga kurang berpengaruh terhadap konsumsi daya. Dari hasil ini, fitur **Global_active_power, Global_intensity, dan Sub_metering_3** lebih relevan untuk analisis lebih lanjut. Oleh karena itu, **Sub_metering_3** dapat dijadikan indikator utama dalam memahami pola penggunaan listrik dalam dataset ini.

2. Training Model

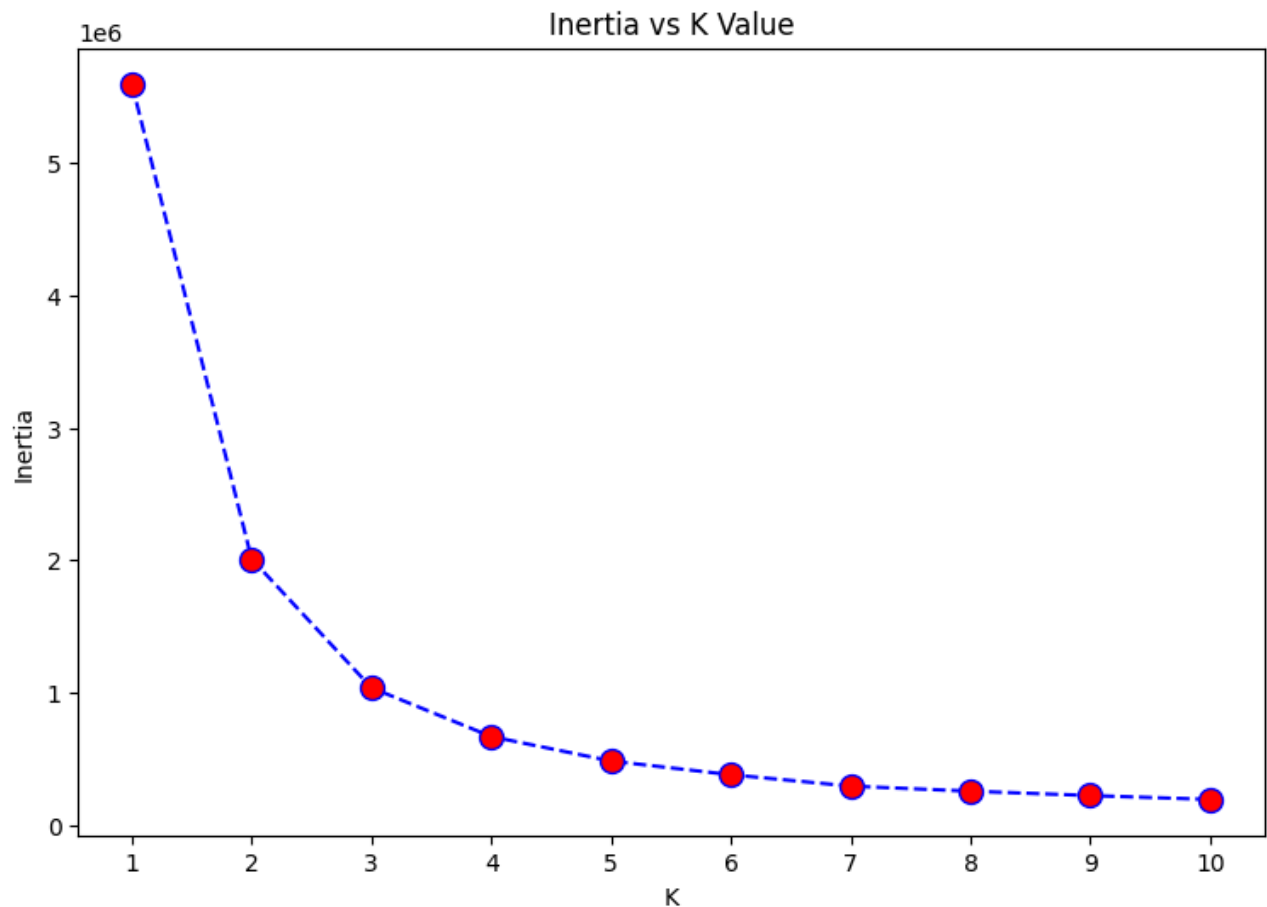
- Elbow Method

```
In [ ]: from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

#Memilih kolom untuk dianalisis
df_subset = df[['Global_active_power', 'Global_intensity', 'Sub_metering_

inertia_list = []
for i in range(1, 11): #menguji dari 1-10 kluster
    kmeans = KMeans(n_clusters=i, random_state=42, n_init=10)
    kmeans.fit(df_subset)
    inertia_list.append(kmeans.inertia_)

#Visualisasi Elbow Method
plt.figure(figsize=(9,6))
plt.plot(range(1, 11), inertia_list, color='blue', linestyle='dashed', ma
plt.title('Inertia vs K Value')
plt.xticks(range(1, 11, 1))
plt.xlabel('K')
plt.ylabel('Inertia')
plt.show()
```



- Silhouette Score

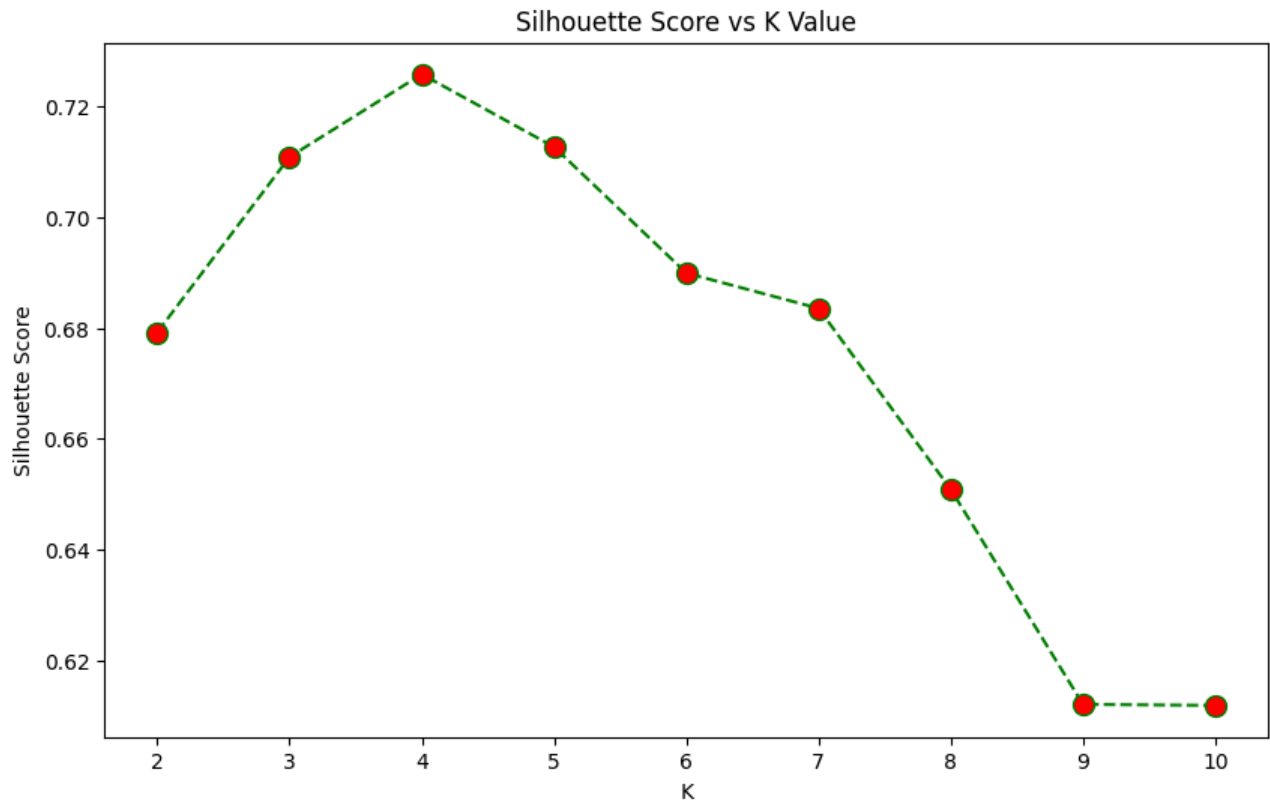
```
In [ ]: import matplotlib.pyplot as plt
import pandas as pd
import numpy as np
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

#Mengambil 100000 data pertama atau sampling jika data melebihi batas
df_limited = df_subset.sample(n=min(10000, len(df_subset)), random_state=

silhouette_scores = []
for k in range(2, 11): #minimal 2 kluster
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = kmeans.fit_predict(df_limited)
    silhouette_scores.append(silhouette_score(df_limited, labels))

#Visualisasi Silhouette Scores
plt.figure(figsize=(10, 6))
plt.plot(range(2, 11), silhouette_scores, color='green', linestyle='dashed',
         marker='o', markerfacecolor='red', markersize=10)
plt.title('Silhouette Score vs K Value')
plt.xlabel('K')
```

```
plt.ylabel('Silhouette Score')
plt.show()
```

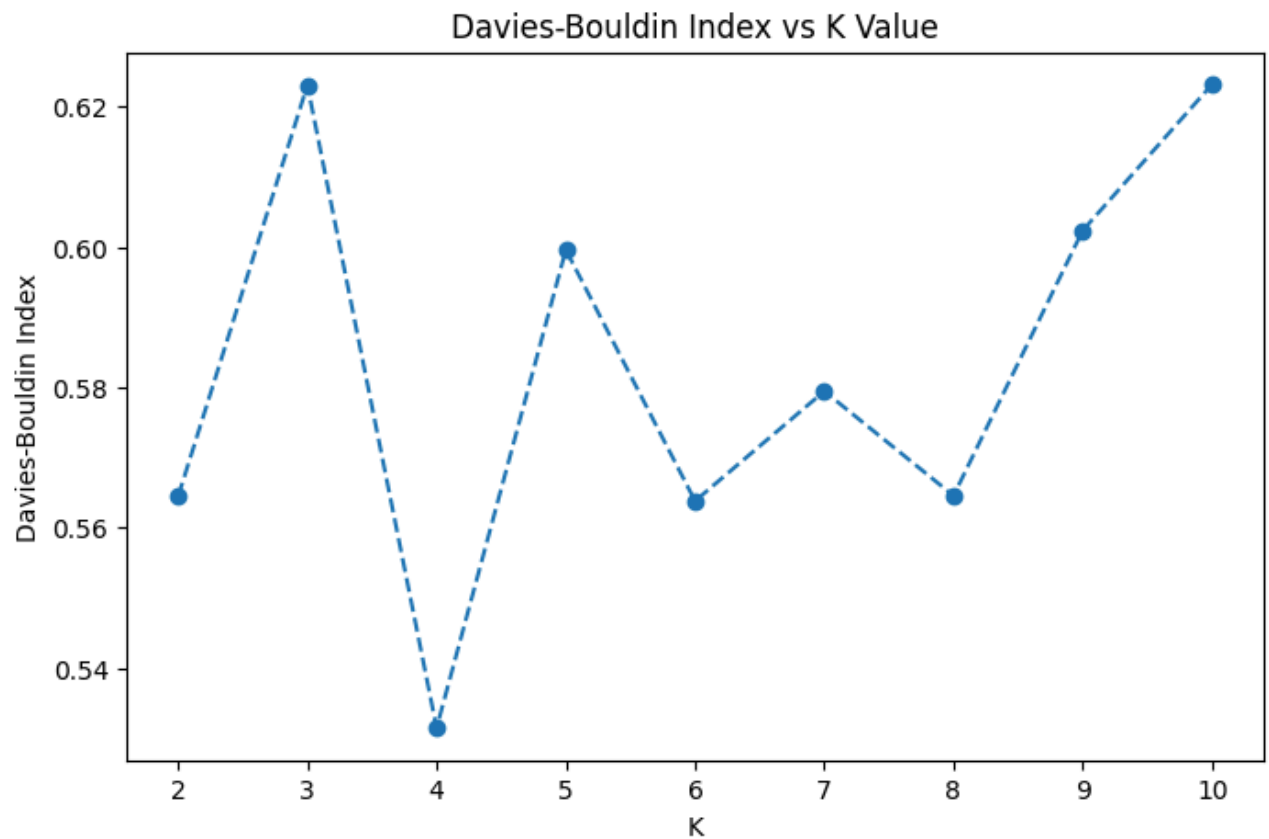


- Davies-Bouldin Index

```
In [ ]: from sklearn.metrics import davies_bouldin_score

dbi_scores = []
for k in range(2, 11): #minimal 2 kluster
    kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
    labels = kmeans.fit_predict(df_subset)
    dbi = davies_bouldin_score(df_subset, labels)
    dbi_scores.append(dbi)

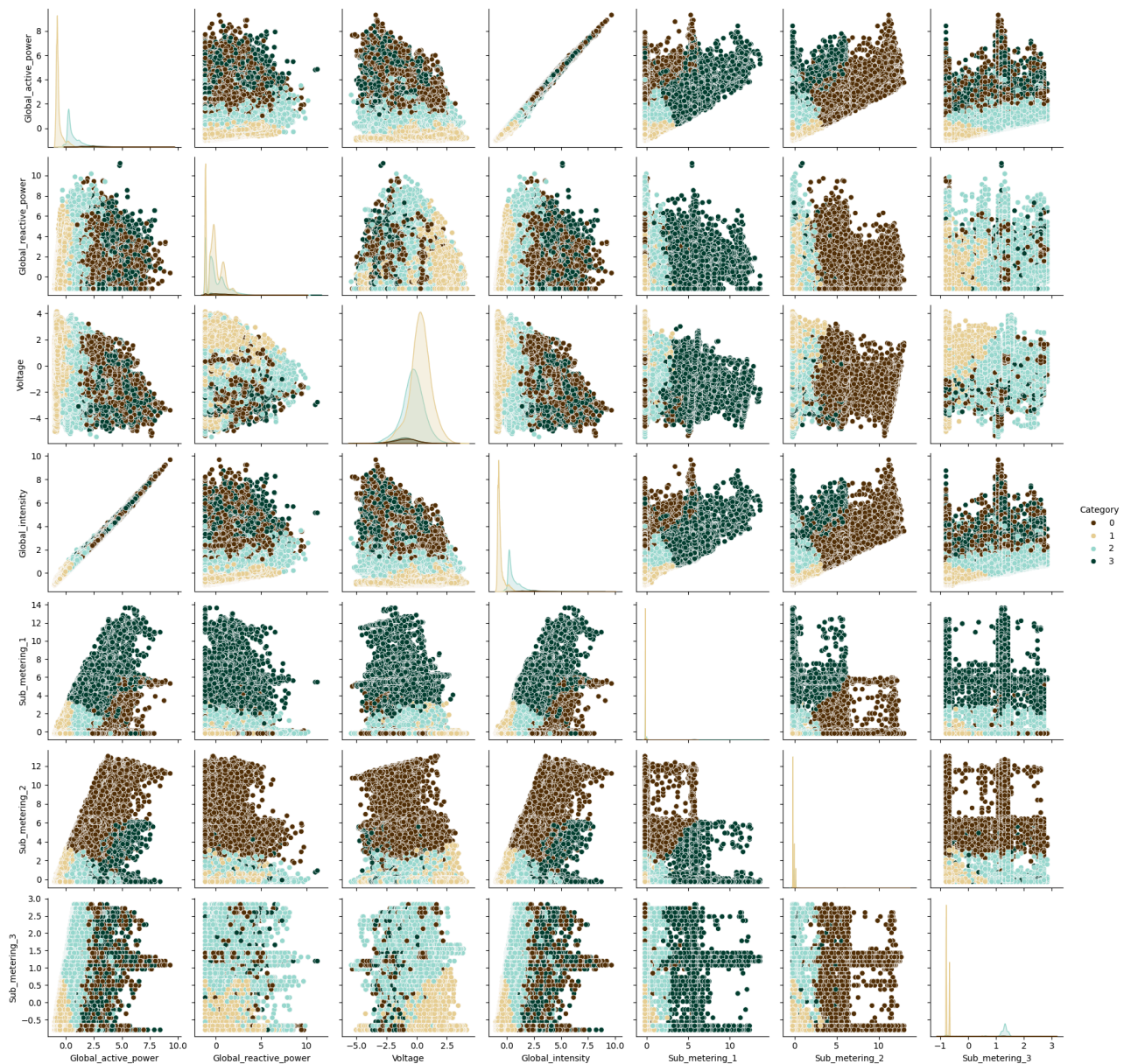
#Visualisasi Davies-Bouldin Index
plt.figure(figsize=(8,5))
plt.plot(range(2, 11), dbi_scores, marker='o', linestyle='--')
plt.xlabel('K')
plt.ylabel('Davies-Bouldin Index')
plt.title('Davies-Bouldin Index vs K Value')
plt.show()
```



- K-Means Clustering

```
In [ ]: km = KMeans (n_clusters = 4, random_state = 42, n_init = 10).fit(df)
df["Category"] = km.labels_
sns.pairplot(data = df, hue = "Category", palette = "BrBG")
```

```
Out[ ]: <seaborn.axisgrid.PairGrid at 0x79f0e7a0af90>
```



Hasil Analisis Klasterisasi K-Means

Proses klasterisasi menggunakan algoritma K-Means dengan jumlah klaster sebanyak 4 menghasilkan pembagian data pengguna listrik menjadi empat segmen utama berdasarkan karakteristik konsumsi energi.

- **Klaster 0:** Pengguna dengan pemakaian energi paling rendah. Kelompok ini menunjukkan pola penggunaan yang konsisten dan tidak fluktuatif, mencerminkan rumah tangga yang hemat energi dan hanya menggunakan peralatan listrik dasar.

- Klaster 1: Pengguna dengan tingkat konsumsi rendah hingga sedang. Terdapat sedikit variasi dalam pemakaian energi, yang menunjukkan penggunaan beberapa alat rumah tangga tambahan seperti kulkas atau televisi, namun masih tergolong efisien.
- Klaster 2: Pengguna dengan tingkat konsumsi sedang hingga tinggi. Rumah tangga dalam klaster ini kemungkinan lebih aktif dalam menggunakan berbagai peralatan elektronik, termasuk perangkat yang menyedot daya cukup besar seperti mesin cuci atau pemanas.
- Klaster 3: Pengguna dengan konsumsi listrik tertinggi dan pola yang bervariasi. Kemungkinan besar, kelompok ini mencakup rumah dengan banyak anggota, penggunaan alat listrik dalam jumlah besar, atau bahkan tempat usaha skala kecil.

Setiap klaster memperlihatkan kecenderungan unik dalam pola konsumsi energi, yang dapat dimanfaatkan untuk strategi efisiensi atau segmentasi kebijakan energi yang lebih tepat sasaran.